

Regular Common Questions

1. a)(2019) Define data structure. Why data Structure is necessary?

Ans:

A data structure is a fundamental concept in computer science, representing a systematic way to organize and store data in a computer's memory. It defines the format and arrangement of data elements, allowing for efficient access, retrieval, and manipulation of information. Data structures serve as building blocks for designing algorithms and solving various computational problems.

Data structures are necessary for several reasons:

Efficiency: They enable the optimization of algorithms, ensuring that operations on data can be performed quickly and with minimal resource consumption.

Abstraction: Data structures abstract the complexities of data management, providing a clear interface for programmers to work with, irrespective of the underlying implementation details.

Reusability: Well-designed data structures can be reused across different applications, saving time and effort in software development.

Scalability: They can adapt to handle large datasets and dynamic changes, making them crucial for robust and adaptable software systems.

In summary, data structures are essential tools in computer science, facilitating efficient data management, algorithm design, and problem-solving.

1. b)(2019) What do you mean by Linear data Structure and Nonlinear data structure? Give an example.

Ans:

Linear data structures and nonlinear data structures are two fundamental categories of data structures in computer science, each with distinct characteristics and use cases.

Linear Data Structure: Linear data structures organize data elements sequentially, with each element connected to the one before and after it. Elements are accessed in a linear, one-dimensional fashion. Examples of linear data structures include:

a. Arrays: A collection of elements stored in contiguous memory locations. Elements are accessed by their index. For example, an integer array `arr[5]` contains five elements accessed as `arr[0]`, `arr[1]`, ..., `arr[4]`.

b. Linked Lists: A chain of nodes, where each node contains a data element and a reference (or pointer) to the next node. Linked lists can be singly linked (each node points to the next) or doubly linked (each node points to both the next and the previous).

Or,

Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list.

c. Stacks: A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. Think of it as a stack of plates where you can only add or remove plates from the top.

d. Queues: A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle, which means that the element that is added first to the queue will be the first one to be removed.

Nonlinear Data Structure: Nonlinear data structures do not have a sequential arrangement of elements. Elements can be interconnected in various ways, creating complex relationships. Examples include:

a. Trees: Hierarchical structures where each element (node) has one parent and zero or more children. Common tree types include binary trees, binary search trees, and balanced trees like AVL and Red-Black trees.

b. Graphs: Collections of nodes (vertices) connected by edges. Graphs can be directed (edges have a direction) or undirected. They are used to represent complex relationships, networks, and various real-world scenarios.

1. a)(2020) How can you differ linear data structures from non-linear data structures?

Ans:

Linear data structures and non-linear data structures differ in how they organize and connect data elements:

Linear Data Structures: These structures organize data elements sequentially, with each element having a unique predecessor and successor. Elements are accessed in a linear, one-dimensional manner. Examples include arrays, linked lists, stacks, and queues. Linear structures are suitable for tasks where data follows a straightforward order, such as ordered lists or simple data storage.

Non-linear Data Structures: Non-linear structures do not have a sequential arrangement. Elements are interconnected in complex ways, forming hierarchical or interconnected relationships. Examples include trees and graphs. Non-linear structures are used when data exhibits complex relationships, like hierarchical data, networks, and intricate dependencies among elements.

1. a)(2021) What is the benefit of binary search over linear search?

Ans:

Aspect	Binary Search	Linear Search
Time Complexity	$O(\log n)$ in the worst case	$O(n)$ in the worst case
Efficiency	Highly efficient for sorted data	Linearly searches through data
Comparison	Divides search space in half with each comparison	Performs linear comparisons
Sorted Data Requirement	Requires data to be sorted	Works with unsorted data as well

Use Cases	Ideal for large datasets where quick retrieval is essential	Suitable for small lists or unsorted data
Example	Searching in a phone directory	Scanning through an unsorted list

2. b)(2021) Briefly discuss the terms garbage collection, overflow and Underflow.

Ans:

Garbage Collection: Garbage collection is an automatic memory management process in programming languages like Java and C#. It identifies and reclaims memory occupied by objects that are no longer accessible, preventing memory leaks and optimizing memory usage.

Overflow: Overflow occurs when a data type's value exceeds its maximum representable limit. For example, if an integer variable meant to store values between -32,768 and 32,767 exceeds this range, an overflow occurs, often resulting in unexpected behavior or data corruption.

Underflow: Underflow is the opposite of overflow and happens when a data type's value falls below its minimum representable limit. For instance, if a floating-point number intended to store small values becomes too small to be represented accurately, an underflow occurs, potentially leading to loss of precision.

1. b)(2020) Simulate the binary search algorithm on the following data: 10 30 50 78 90 95 100 105 110 (suppose we search for item 110).

Ans:

Here's a simplified simulation of the binary search algorithm to find the item 110 in the given sorted data:

1. Start with the entire sorted list: [10, 30, 50, 78, 90, 95, 100, 105, 110].
2. Find the middle element: In this case, it's 90.
3. Compare 90 with the target value, 110:
 - Since $90 < 110$, eliminate the left half of the list.
4. New list to search: [100, 105, 110].
5. Find the middle element: 105.
6. Compare 105 with the target value, 110:
 - Since $105 < 110$, eliminate the left half.
7. New list to search: [110].
8. Find the middle element: It's 110.
9. Compare 110 with the target value, 110:
 - We've found the target value.

The binary search algorithm successfully located the item 110 in the list.

Array and matrices

1. c)(2019) What is 2D array? How can you represent 2D array in memory?

Ans:

A 2D array, also known as a two-dimensional array, is a data structure that organizes data elements in a grid or matrix-like format with rows and columns. It's used to represent tables, matrices, or grids of values in programming. Each element in a 2D array is uniquely identified by its row and column indices.

In memory, a 2D array is typically represented as a contiguous block of memory, with elements arranged in rows and columns. The elements are stored in a sequential manner, row by row or column by column, depending on the programming language and memory layout. The memory is organized such that each row is stored consecutively, ensuring efficient access to individual elements through row and column indexing.

1. c)(2020) How are the elements of a 2D array stored in the memory?

Ans:

In memory, the elements of a 2D array are stored in a contiguous block, organized in a row-major or column-major fashion, depending on the programming language and memory layout.

Row-major order: In this arrangement, the elements of each row are stored consecutively in memory. This means that elements within the same row are adjacent in memory, while elements in different rows are separated. Accessing elements by row is efficient, as they are stored together.

Column-major order: Here, elements of each column are stored sequentially. Elements within the same column are adjacent in memory, simplifying column-wise access.

The choice of storage order can affect access performance, especially when traversing the array. Most programming languages, like C and C++, use row-major order by default, while others, like Fortran, use column-major order. The choice depends on the language's conventions and the specific requirements of the application.

2. b)(2020) What is sparse matrix? What is the difference between triangular matrix and tridiagonal matrix?

Ans:

Sparse Matrix: A sparse matrix is a matrix in which most of the elements are zero, resulting in a data structure that efficiently stores and manipulates only the non-zero elements to conserve memory and processing resources.

Characteristic	Triangular Matrix	Tridiagonal Matrix
Definition	A matrix in which all elements above or below the main diagonal are zero.	A matrix in which all elements outside the main diagonal and its adjacent diagonals (above and below) are zero.
Non-zero Elements	Main diagonal and either above or below it contain non-zero elements.	Main diagonal and its two adjacent diagonals contain non-zero elements; all other elements are zero.
Example	$\begin{bmatrix} a & 0 & 0 \\ b & c & 0 \\ d & e & f \end{bmatrix}$	$\begin{bmatrix} a & b & 0 & 0 \\ c & d & e & 0 \\ 0 & f & g & h \\ 0 & 0 & i & j \end{bmatrix}$
Number of Non-zero Diagonals	One (main diagonal)	Three (main diagonal and its two adjacent diagonals)

Application	Often used for systems with triangular symmetry, such as solving systems of linear equations.	Frequently used in numerical analysis for solving differential equations and finite difference methods.
--------------------	---	---

1. c)(2021) We often store sparse matrices in a 1D array to save spaces. What is the memory saving if we store a sparse matrix in a 1D array rather than a 2D array?

Ans:

Storing a sparse matrix in a 1D array can save a significant amount of memory compared to using a 2D array. The exact amount of memory saved depends on the sparsity of the matrix, which is the proportion of zero elements to non-zero elements.

In a 2D array, memory is allocated for every element, whether it's zero or non-zero. In contrast, when using a 1D array to store a sparse matrix, you only need to store the non-zero elements along with their row and column indices. Therefore, the memory saving can be substantial, especially when the matrix is highly sparse and contains mostly zero elements.

The greater the sparsity, the more memory you save by using a 1D array representation.

2. b)(2019) What is pointer array? How a pointer can save memory space when storing a 2D array?

Ans:

Pointer Array: A pointer array is an array in which each element is a pointer to another data structure or value. In the context of a 2D array, a pointer array is used to efficiently represent a matrix. Instead of allocating memory for the entire 2D array, a pointer array consists of pointers (addresses) to individual rows or arrays, and each row can have a different length.

Using a pointer array can save memory space when storing a 2D array because:

1. It allows you to allocate memory only for the necessary rows, rather than allocating a fixed-size rectangular block for the entire 2D array.
2. Different rows can have different lengths, which is useful for representing irregularly sized matrices or sparse data efficiently.

By dynamically allocating memory for each row based on its actual size, you conserve memory resources, especially when dealing with large and sparsely populated 2D data structures.

2. c)(2019) Is there any differences between a record and a linear array?

Ans:

Characteristic	Linear Array	Record
Structure	One-dimensional, sequence of elements	Composite, stores multiple fields
Elements	Homogeneous (usually of the same data type)	Heterogeneous (different data types for each field)
Access	By index (position)	By field name or label
Use Cases	Lists of similar elements	Representing structured data entities, entities with multiple properties
Example	[1, 2, 3, 4, 5]	{Name: "John", Age: 30, Address: "123 Main St"}

This table highlights the distinctions between these two data structures, emphasizing their respective purposes and characteristics.

2. c)(2020) How can you locate element a_{if} of a sparce matrix from a ID array?

Ans:

Depending on the ordering (row-major or column-major), you can calculate the index in the 1D array corresponding to element a_{if} using the formula:

- Row-Major Order: Index = $(i * \text{num_columns}) + j$
- Column-Major Order: Index = $(j * \text{num_rows}) + i$

1. b)(2021) Maze(1:6,-4:1,5:10) is a 3D array with base=100, w=4,calculate Maze[3,-2,8] address in a row major order and column major order.

Ans:

Address = base + $((i * \text{n_columns} + j) * \text{n_slices} + k) * w$

Address = $100 + ((3 * 11 + (-2)) * 6 + 8) * 4$

Address = $100 + ((33 - 2) * 6 + 8) * 4$

Address = $100 + (31 * 6 + 8) * 4$

Address = $100 + (186 + 8) * 4$

Address = $100 + 194 * 4$

Address = $100 + 776$

Address = 876

2. a)(2019) Suppose Book (2:10, -4:-1, 5:8) is a 3D array with base = 200, w=4, calculate Book[5,-1,7] address in a row major order and column major order.

Ans:

Address = base + ((i * n_columns + j) * n_slices + k) * w

Address = 200 + ((5 * 9 - 1) * 4 + 7) * 4

Address = 200 + ((45 - 1) * 4 + 7) * 4

Address = 200 + (44 * 4 + 7) * 4

Address = 200 + (176 + 7) * 4

Address = 200 + 183 * 4

Address = 200 + 732

Address = 932

Linked List

2. a)(2021) What is a linked list? Explain the main differences between the linked list and linear array?

Ans:

Linked Lists: A chain of nodes, where each node contains a data element and a reference (or pointer) to the next node. Linked lists can be singly linked (each node points to the next) or doubly linked (each node points to both the next and the previous).

Or,

Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list.

Characteristic	Linked List	Linear Array (Array)
Structure	Consists of nodes, each with data and a reference to the next node.	Contiguous block of memory with elements indexed by position.
Dynamic Size	Dynamic; can easily grow or shrink by adding or removing nodes.	Static; size is fixed at the time of declaration.
Memory Allocation	Non-contiguous; nodes can be scattered in memory.	Contiguous; elements stored sequentially in memory.
Access Time	Requires traversal from one node to another.	Direct access using index (position).
Insertions/Deletions	Efficient for insertions and deletions within the list.	Less efficient, especially for middle insertions/deletions.

Memory Efficiency	No wasted memory for unused space in the list.	May waste memory if not fully utilized.
--------------------------	--	---

2. c)(2021) One of the advantages of linked lists is the ability to insert data into the list easily. Explain with your own words and figures how to insert data at the beginning, after a given node, at the end and to a sorted list.

Ans:

Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list. One of the advantages of linked lists is their ability to easily insert data at various positions within the list.

Inserting Data at the Beginning:

To insert data at the beginning of a linked list, a new node is created with the desired value. The next pointer of this new node is then set to point to the current head of the list. Finally, the head pointer is updated to point to the newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> NodeN

After insertion: NewNode -> Head -> Node1 -> Node2 -> ... -> NodeN

Inserting Data after a Given Node: To insert data after a specific node in a linked list, we need to locate that node first. Once we have found the desired node, we create a new node with the desired value and update its next pointer to point to the next node in the original list. Then, we update the next pointer of the given node to point to our newly inserted node.

Before insertion: Head -> Node1 -> GivenNode -> Node2 -> ... -> NodeN

After insertion: Head -> Node1 -> GivenNode -> NewNode -> Node2 -> ... -> NodeN

Inserting Data at the End: To insert data at the end of a linked list, we need to traverse through the entire list until we reach the last node. Once we have reached the last node, we create a new node with the desired value and set its next pointer to null. Then, we update the next pointer of the current last node to point to our newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> NodeN -> LastNode

After insertion: Head -> Node1 -> Node2 -> ... -> NodeN -> LastNode -> NewNode

Inserting Data into a Sorted List: Inserting data into a sorted linked list requires finding the correct position for the new node based on its value. We start by traversing the list from the head until we find a node whose value is greater than the value we want to insert. Once we have found the appropriate position, we create a

new node with the desired value and update its next pointer to point to the next node in the original list. Then, we update the next pointer of the previous node to point to our newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> PrevNode -> NextNode -> ... -> LastNode

After insertion: Head -> Node1 -> Node2 -> ... -> PrevNode -> NewNode -> NextNode -> ... -> LastNode

3. a)(2020) Mention some differences between array and linked lists.
Ans: Answer in **2. a)(2021)**.

3. b)(2020) Can binary search algorithm be employed on data stored in linked list? Justify your answer.

Ans:

Yes, the binary search algorithm can be employed on data stored in a linked list, but with certain limitations and modifications. Binary search is a divide-and-conquer algorithm that requires random access to elements, which is not possible in a linked list. However, by using a modified approach called "binary search on a sorted linked list," it is still possible to perform a binary search.

In this modified approach, instead of accessing elements directly through indexing, we traverse the linked list by comparing the target value with the middle element of the current sublist. Based on the comparison, we can determine whether to continue searching in the left or right half of the sublist. By repeatedly dividing the sublist in half, we can efficiently locate the target element.

However, it is important to note that binary search on a linked list has some drawbacks compared to an array-based implementation. The time complexity increases from $O(\log n)$ to $O(n)$, as we need to traverse the list linearly. Additionally, random access is not possible, affecting efficiency.

3. c)(2020) Briefly discuss inserting mechanism of an item at the beginning, after a given node, at the end and also to a sorted list.
Ans: Answer in **2. c)(2021)**

3. a)(2019) What are the advantages of Linked List over array?
Ans:

Advantage	Linked List	Array
-----------	-------------	-------

Dynamic Size	Can grow or shrink dynamically.	Has a fixed size determined at creation.
Efficient Insertions/Deletions	Efficient for inserting/deleting elements, especially in the middle.	Insertions/deletions may require shifting, less efficient.
Memory Efficiency	Uses memory proportional to the number of elements.	May waste memory due to preallocated space.
No Need for Preallocation	No need to preallocate memory for a maximum size.	Requires preallocation of a fixed size.
No Wasted Memory	No wasted memory for unused space.	May have unused or uninitialized elements.
Flexibility	Suited for dynamic data structures.	Fixed size can limit flexibility.

3. b)(2019) Briefly discuss inserting mechanism of an item at the beginning, after a given node and at the end.

Ans: Answer in **2. c)(2021) And 3. c)(2020)**

3. c)(2019) What is two way lists? Why it is important? Explain with Schematic diagram.

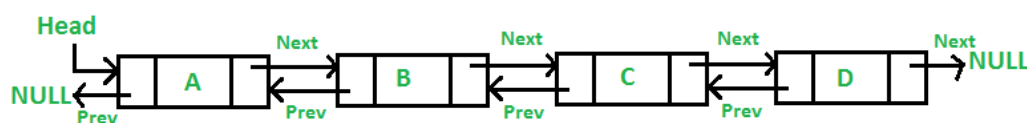
Ans: A two-way linked list, also known as a doubly linked list, is a variation of a linked list where each node has two pointers: one pointing to the next node and another pointing to the previous node. This bidirectional linkage allows for more flexible traversal in both directions, from the head to the tail and vice versa.

Importance of Two-Way Linked Lists:

Bidirectional Traversal: Doubly linked lists allow efficient traversal in both forward and backward directions, making them valuable for tasks like reverse traversal.

Insertions and Deletions: They are efficient for inserting and deleting nodes at both the beginning and end of the list, as well as within the list.

Applications: Doubly linked lists are used in various applications, such as implementing undo/redo functionality in software, navigating browser history, and managing data structures like deque (double-ended queue).



Stack Queue

3. a)(2021) Define stack. Explain the usage of stack in recursive algorithm implementation.

Ans:

Stack: A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. Think of it as a stack of plates where you can only add or remove plates from the top.

Usage in Recursive Algorithm Implementation:

- Stacks are crucial in implementing recursive algorithms, which are algorithms that call themselves to solve a problem.
- When a function in a recursive algorithm is called, its local variables and execution context are pushed onto the stack.
- Each recursive call creates a new stack frame, allowing the algorithm to remember its state at different levels of recursion.
- When the base case of the recursion is reached, the stack starts popping frames, effectively "unwinding" the recursion and allowing the algorithm to compute the final result by combining the results from each level.
- Without a stack, recursive algorithms would not be able to keep track of multiple levels of recursion and could lead to inefficient or incorrect solutions.

3. b)(2021) Simulate the infix to postfix transformation algorithm for $Q:A+(B*C-(D/E\uparrow F)*G)*H$ by showing the stack's contents as each element is scanned.

Ans:

3. c)(2021) Write down a routine to insert an element onto a queue.

Ans:

To insert an element onto a queue, you typically perform the following steps:

Create the Element: Create the element (or data) that you want to insert into the queue.

Allocate Memory (if needed): If you're using a dynamically allocated data structure (e.g., linked list for the queue), allocate memory for the new element.

Set the Value: Assign the value or data to the newly created element.

Update Pointers (if needed): If you're using a linked list to implement the queue, you'll need to update pointers to link the new element to the existing elements in the queue.

5. a)(2020) What is a queue, how is it different from a stack and how is it implemented?

Ans:

A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle, which means that the element that is added first to the queue will be the first one to be removed.

Characteristic	Queue	Stack
Data Structure	Follows FIFO (First-In, First-Out)	Follows LIFO (Last-In, First-Out)
Order of Processing	Oldest element processed first	Newest element processed first
Operations	- Enqueue (Add to rear)	- Push (Add to top)
	- Dequeue (Remove from front)	- Pop (Remove from top)
Implementation	Typically implemented using	Typically implemented using
	arrays or linked lists.	arrays or linked lists.
Common Applications	Task scheduling, print spooling,	Function call execution,
	breadth-first search,	expression evaluation,
	process management.	backtracking algorithms.

Queue Implementation:

- Queues can be implemented using arrays or linked lists.
- In an array-based implementation, you would use two pointers, one pointing to the front and the other to the rear. Enqueue adds elements to the rear, and dequeue removes elements from the front.
- In a linked list-based implementation, each node contains the data and a pointer to the next node. The front and rear are represented by pointers to the first and last nodes, respectively.

Stack Implementation:

- Stacks can also be implemented using arrays or linked lists.
- In an array-based implementation, a pointer (top) keeps track of the current top element. Push adds elements to the top, and pop removes elements from the top.
- In a linked list-based implementation, each node contains the data and a pointer to the next node. The top is represented by a pointer to the first node. Push and pop operations update the top pointer.

5. b)(2020) Write the steps involved in the insertion and deletion of an element in a stack.

Ans:

Here are the simplified steps for inserting (pushing) and deleting (popping) an element in a stack:

Insertion (Push) in a Stack:

Prepare the Element: Get the element you want to add to the stack.

Find the Top: Identify the top of the stack (the last element added).

Place the Element: Put the new element on top of the stack.

Update the Top: Update the top to point to the newly added element.

Deletion (Pop) from a Stack:

Check if the Stack is Empty: Before popping, make sure the stack is not empty.

Access the Top Element: If the stack is not empty, take the element from the top.

Remove the Top Element: Remove the top element from the stack.

Update the Top: Update the top to point to the next element below (if any).

These steps ensure that elements are added and removed in a Last-In, First-Out (LIFO) order, similar to a stack of plates where you add and remove plates from the top.

5. c)(2020) Simulate the postfix expression evaluation algorithm using 18,6,/,6,2,/,+,3,-,4 * by showing stack's contents as each element is scanned.

Ans:

5. a)(2019) Differentiate between stack and queue.

Ans: Answer in **5. a)(2020)**.

5. b)(2019) Convert the following infix expression to its equivalent prefix and postfix expression. $A+(B*C/5-(D/E\uparrow F)*G)*H$

Ans:

5. c)(2019) Simulate the postfix expression evaluation algorithm using 18, 6, /, 6, 2, +, *, 12, 4, /, - by showing stack's contents as each element is scanned.

Ans:

5. d)(2019) What is deque?

Ans: Answer in **5. a)(2020)**.

Hash

4. a)(2021) What is a hash function?

Ans:

A hash function is a mathematical function that takes an input (or 'key') and produces a fixed-size string of characters, which is typically a hexadecimal number or a sequence of digits and letters.

4. b)(2021) The keys 14, 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and plus 3 probing. What is the resultant hash table?

Ans:

4. c)(2021) A hash table of length 10 uses open addressing with hash function $h(k) = k \bmod 10$, and linear probing. After inserting 6 values into an empty has table, the table is as shown below.

		42	23	34	52	46	33		
0	1	2	3	4	5	6	7	8	9

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- (A) 46, 42, 34, 52, 23, 33
- (B) 34, 42, 23, 52, 33, 46
- (C) 46, 34, 42, 23, 52, 33
- (D) 42, 46, 33, 23, 34, 52

Ans:

We will check whether sequence given in option A can lead to hash table given in question.

Option A inserts 46, 42, 34, 52, 23, 33 as:

For key 46, $h(46)$ is $46\%10 = 6$. Therefore, 46 is placed at 6th index in the hash table.

For key 42, $h(42)$ is $42\%10 = 2$. Therefore, 42 is placed at 2nd index in the hash table.

For key 34, $h(34)$ is $34\%10 = 4$. Therefore, 34 is placed at 4th index in the hash table.

For key 52, $h(52)$ is $52\%10 = 2$. However, index 2 is occupied with 42. Therefore, 52 is placed at 3rd index in the hash table. But in given hash table, 52 is placed at 5th index.

Therefore, sequence in option A can't generate hash table given in question.

In the similar way, we can check for other options as well which leads to answer as (C).

Here's the current state of the hash table after inserting the value

4. a)(2019) Briefly discuss different hash collision resolution techniques.

4. b)(2019) Draw a hash table with open addressing and a size of 9. Use the hash function " $k\%9$ " and quadratic probing. Insert the keys: 5, 29, 20, 0, 27, 26, 17 and 18 into your table.

Ans:

In quadratic probing insertion is :

0	0
1	27
2	29
3	20
4	
5	5
6	17
7	18
8	26

4. c)(2019) Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function $x \bmod 10$, which of the following statements are true? Explain.

- 9679, 1989, 4199 hashes to the same value
- 1471, 6171 has to the same value
- All elements has to the same value
- Each element hashes to a different value
- None of the above.

Ans:

- i) is true because modding 9679, 1989, 4199 hashes to the same value.
- ii) is also true because they hashes to same value.
- iii) is false because some have difference hash value.
- iv) is false because some have same value.
- v) is false because already i and ii is true.

Tree

5. a)(2021) Define the terms i) siblings, ii) ancestor, and iii) depth of a binary tree.

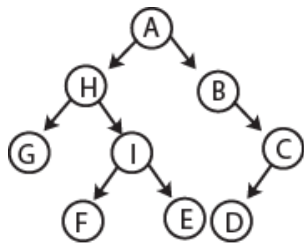
Ans:

i) Siblings: Siblings are nodes in a binary tree that share the same parent node. In other words, two nodes are siblings if they are both children of the same parent node.

ii) Ancestor: An ancestor of a node in a binary tree is any node that lies on the path from the root node to that particular node. Ancestor nodes are higher in the tree hierarchy and can be found by tracing the path from the root to the target node.

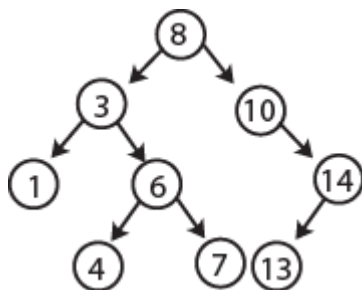
iii) Depth: The depth of a node in a binary tree represents its level within the tree hierarchy. The root node is at depth 0, and each level below it increases the depth by 1. So, the depth of a node indicates how many edges or levels are between it and the root node.

5. b)(2021) Tree traversal (also known as walking the tree) refers to the process of visiting each node exactly once. Simulate the preorder traversal algorithm for the following tree.



Ans:

6. c)(2021) How can you search '5' in the following binary search tree? However if '5' is not in the tree just insert to its appropriate place and show the resultant tree.



Ans:

8. a)(2021) Suppose we want to encode a message constructed from the symbols A, B, C, D, E, F and G using a fixed-length code. How many bits are required to encode the message FDEGAACAAGAAAFABA?

Ans:

8. b)(2021) Build the Huffman coding tree for the message 'science engineering'.

Ans:

8. c)(2021) Suppose you are to insert a node 'X' as the right child of node 'P'. Discuss the inserting mechanisms with figures.

- i. When the right subtree of 'P' is empty.
- ii. When the right subtree of 'P' is not empty.

Solve the separable differential equation

$$(x - 4)y^4 dx - x^3(y^2 - 3)dy = 0$$

Ans:

To insert a node 'X' as the right child of node 'P' in a binary tree, there are two scenarios to consider:

- i. When the right subtree of 'P' is empty: In this case, you can simply create a new node 'X' and set it as the right child of 'P'. Here's a
- ii. When the right subtree of 'P' is not empty: In this scenario, you'll need to push the existing right subtree down one level by making 'X' the new right child of 'P' and attaching the previous right subtree as the right child of 'X'.

Rearrange the terms to separate variables: $(x - 4)y^4 dx = x^3(y^2 - 3) dy$

Divide both sides by $(x - 4)y^4$ and multiply both sides by dx : $dx = (x^3(y^2 - 3))/(x - 4)y^4 dy$

Now, separate variables by moving dy to one side and dx to the other side: $(x - 4)y^4 dy = (x^3(y^2 - 3)) dx$

Divide both sides by $(x^3(y^2 - 3))$ to isolate dy on the left side and dx on the right side: $(x - 4)y^4 / (x^3(y^2 - 3)) dy = dx$

Now, we can integrate both sides:

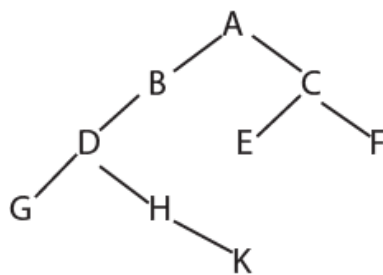
$$\int (x - 4)y^4 / (x^3(y^2 - 3)) dy = \int dx$$

6. a)(2019) What is complete binary tree? What is the parent child relationship?

6. b)(2019) Discuss the linked representation of binary tree in Memory.

6. a)(2020) When do we call a binary tree to be a 2-tree? What is the depth of a complete binary tree with 10,00,000 nodes?

6. b)(2020) Traverse the following tree in preorder, inorder and postorder.



6. c)(2020) How can binary search tree overcome the disadvantages of array and linked list? Explain the process of inserting a new unique item in a binary search tree with a simple example.

Heap

6. c)(2019) Simulate the maxheap algorithm for the following values: 100, 29, 90, 148, 12, 34, 90, 89, 120, 99 and 12.

Graph

6. a)(2021) What is the adjacency matrix? How is it formed?

6. b)(2021) Define the terms (i) Isolated node, (ii) Simple Path and (iii) Weighted Graph.

7. a)(2021) What is meant strongly and weakly connected in a graph?

7. b)(2021) Prove that the maximum number of edges that a graph with n vertices is $n*(n-1)/2$.

Ans: We have that is a simple graph, no parallel or loop exist. Therefore the degree of each vertex will be one less than the total number of vertices (at most). ie, **degree=n-1**

eg. we have a graph with two vertices (so one edge) degree=**(n-1)**.

$$(n-1)=(2-1)=1$$

We know that the sum of the degree in a simple graph always even ie, **$\sum d(v)=2E$**

here **$d(v)=n-1$** : we have n vertices the total degree is **$n(n-1)$**

$$n*(n-1)=2E$$

$$E=n*(n-1)/2$$

7. c)(2021) Explain Breadth First search algorithm with example.

7. a)(2020) Suppose the following eight numbers are inserted in order into an empty binary search tree : 40, 60, 50, 33, 35, 55, 11, 12. Show the tree as each number is inserted into a binary search tree.

7. b)(2020) Consider the following adjacency matrix below:

$$A = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Now find out A^2 , A^3 , A^4 , B_4 and from that make the path matrix and tell whether this is strongly connected or not.

8. a)(2020) What is directed graph?

8. b) (2020) Discuss the sequential representation of graph with example.

8. c) (2020) Use the Warshall's algorithm to find the shortest path matrix of the adjacency matrix given below:

$$A = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

7. a)(2019) What is Directed Graph? Explain.

7. b)(2019) Briefly discuss the advantage of binary search tree over array and linked list.

7. c)(2019) Use the Warshall's algorithm to find the shortest path matrix of the weighted matrix given below.

$$W = \begin{pmatrix} 2 & 5 & 0 & 0 \\ 2 & 0 & 1 & 4 \\ 5 & 3 & 3 & 3 \\ 4 & 2 & 3 & 0 \end{pmatrix}$$

8. a)(2019) Discuss the sequential Representation of Graph with example.

8.b)(2019) How many ways a graph G can be traversed? What is the significance of the STATUS field?

8.c)(2019) Consider the adjacency list of the Graph G in the following table. Find the nodes that are reachable from node C using Depth First Search.

Node	Adjacency	Node	Adjacency
A	G,E	E	C
B	C	F	A,B
C	F	G	B,C,E
D	C	H	D

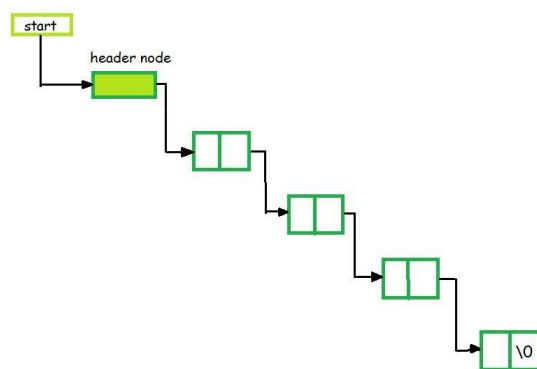
what is header linked list?

A *header node* is a special node that is found at the *beginning* of the list. A list that contains this type of node, is called the header-linked list. This type of list is useful when information other than that found in each node is needed.

Types of Header Linked List(Grounded Header,Circular Header)

1. Grounded Header Linked List

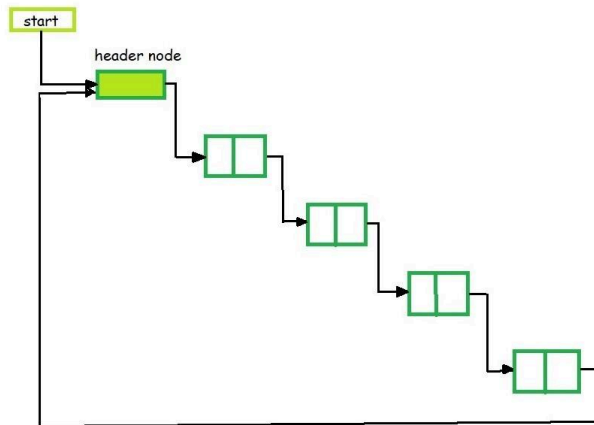
It is a list whose *last node* contains the *NULL* pointer. In the header linked list the **start** pointer always points to the header node. **start** -> **next = NULL** indicates that the grounded header linked list is *empty*. The operations that are possible on this type of linked list are *Insertion, Deletion, and Traversing*.



2.

Circular Header Linked List

A list in which *last node* points back to the *header node* is called circular linked list. The chains do not indicate first or last nodes. In this case, external pointers provide a frame of reference because last node of a circular linked list does **not contain** the **NULL** pointer. The possible operations on this type of linked list are *Insertion, Deletion and Traversing*.



9.What is polish notation?What are the benefits of polish notation?

Ans:

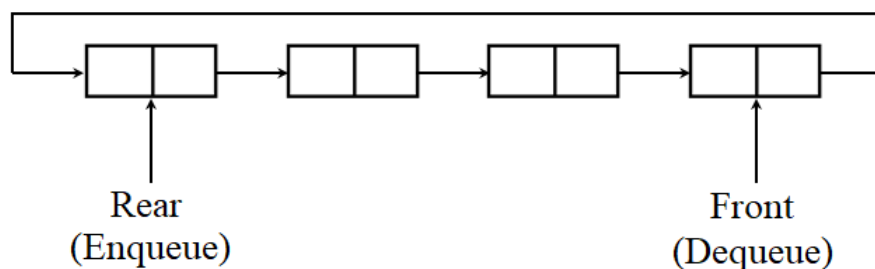
Polish notation, also known as prefix notation, is a mathematical notation in which every operator follows all of its operands. This is in contrast to infix notation, which is the more common notation used in mathematics, where operators are placed between their operands. Polish notation eliminates ambiguity, simplifies parsing, and offers a compact, consistent, and efficient way to represent and evaluate mathematical expressions, making it suitable for programming languages and stack-based systems.

10.Circular Queue. Ans:

A circular queue permits better memory utilization than a simple queue when the queue has a fixed size.

In this queue, the last node points to the first node and creates a circular connection. Thus, it allows us to insert an item at the first node of the queue when the last node is full and the first node is free.

It's also called a ring buffer:

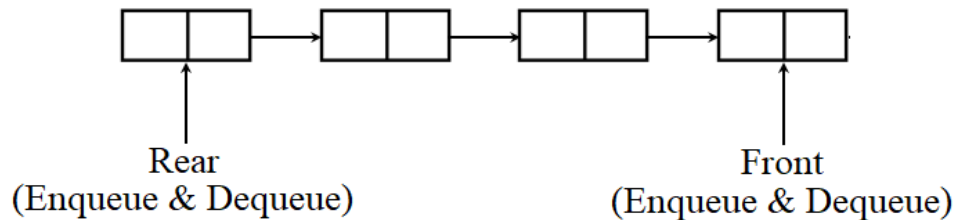


11.Priority Queue. Ans:

A priority queue is a special kind of queue in which each item has a predefined priority of service. In this queue, the enqueue operation takes place at the rear in the order of arrival of the items, while the dequeue operation takes place at the front based on the priority of the items.

12. Double-Ended Queue (Deque). Ans:

A deque is also a special type of queue. In this queue, the enqueue and dequeue operations take place at both front and rear. That means, we can insert an item at both the ends and can remove an item from both the ends. Thus, it may or may not adhere to the FIFO order:



13. What is recursion. Ans:

Recursion is defined as a process which calls itself directly or indirectly and the corresponding function is called a recursive function.

14. explain the use of recursion. Ans:

Recursion is used in many fields of computer science and mathematics, which includes:

- Searching and sorting algorithms: Recursive algorithms are used to search and sort data structures like trees and graphs.
- Mathematical calculations: Recursive algorithms are used to solve problems such as factorial, Fibonacci sequence, etc.
- Compiler design: Recursion is used in the design of compilers to parse and analyze programming languages.
- Graphics: many computer graphics algorithms, such as fractals and the Mandelbrot set, use recursion to generate complex patterns.
- Artificial intelligence: recursive neural networks are used in natural language processing, computer vision, and other AI applications.

Hash function:

1.what is hash function?

A Hash Function is a function that converts a given numeric or alphanumeric key to a small practical integer value. The mapped integer value is used as an index in the hash table. In simple terms, a hash function maps a significant number or string to a small integer that can be used as the index in the hash table.

2.what can be the techniques to avoid collusion?

To avoid collisions in a hash table:

1. ****Use a Good Hash Function****: Select a quality hash function that distributes keys evenly.
2. ****Open Addressing****: Handle collisions by placing the item in the next available slot.

3. **Separate Chaining**: Use linked lists or other data structures to store multiple items in the same hash slot.
4. **Dynamic Resizing**: Rehash the table when it reaches a certain load factor to maintain a balanced table.
5. **Double Hashing**: Combine multiple hash functions to find the next available slot for collisions.
6. **Robin Hood Hashing**: Reorganize elements in case of collisions to minimize the difference in probe lengths.

These techniques reduce the likelihood of collisions and ensure efficient hash table performance.

3.briefly discuss different hash collision resolution techniques

Hash collision resolution techniques include:

1. **Open Addressing**: Linear probing, quadratic probing, and double hashing search for the next available slot when collisions occur.
2. **Separate Chaining**: Slots hold linked lists or data structures to store colliding keys.
3. **Robin Hood Hashing**: Reorganizes elements during collisions to minimize differences in probe lengths.
4. **Cuckoo Hashing**: Uses multiple hash functions and two tables, swapping items between tables upon collisions.
5. **Coalesced Hashing**: Combines open addressing and chaining, allowing limited collisions before using chaining.
6. **Double Hashing**: Utilizes a second hash function to determine probe steps, reducing clustering.
7. **Linear/Extendible Hashing**: Dynamically adjusts the hash table's size to minimize collisions.

Each technique addresses collisions differently, catering to various use cases based on key distribution and performance requirements.

4.Open hashing or chaining.Ans:

Open hashing or more widely known as chaining is one of the simplest approaches to avoid collision in hash tables. In open hashing, each hash table slot, also known as a bucket, contains a linked list of elements that hash to the same slot. The term chaining is used to describe the process because the linked list is used, which is like a chain of elements.

5.Closed Hashing

Like open hashing, closed hashing is also a technique used for collision resolution in hash tables. Unlike open hashing, where collisions are resolved by chaining elements in separate chains, closed hashing aims to store all elements directly within the hash table itself without the use of additional data structures like linked lists.

6.Linear Probing:

In linear probing, if a collision occurs and a slot is already occupied, the algorithm linearly searches for the next available slot in a sequential manner. It checks the next slot and continues checking subsequent slots until it finds an empty slot.

7.Quadratic Probing:

In quadratic probing, instead of probing the next slot linearly, the algorithm uses a quadratic function to determine the next probe position. The probing sequence follows a quadratic pattern until an empty slot is found.

8.Double hashing:

utilizes two hash functions to determine the probe sequence. When a collision occurs, it applies the second hash function to calculate an offset, which is then used to find the next slot to probe.

Tree

1. Root

In a tree data structure, the first node is called as Root Node. Every tree must have root node. We

can say that root node is the origin of tree data structure. In any tree, there must be only one root

node. We never have multiple root nodes in a tree. In above tree, A is a Root node

2. Edge

In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with

'N' number of nodes there will be a maximum of 'N-1' number of edges.

3. Parent

In a tree data structure, the node which is predecessor of any node is called as PARENT NODE. In

simple words, the node which has branch from it to any other node is called as parent node.

Parent

node can also be defined as "The node which has child / children". e.g., Parent (A,B,C,D).

4. Child

In a tree data structure, the node which is descendant of any node is called as CHILD Node. In

simple words, the node which has a link from its parent node is called as child node. In a tree, any

parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.

e.g., Children of D are (H, I,J).

5. Siblings

In a tree data structure, nodes which belong to same Parent are called as SIBLINGS. In simple

words, the nodes with same parent are called as Sibling nodes. Ex: Siblings (B,C, D)

6. Leaf

In a tree data structure, the node which does not have a child (or) node with degree zero is called

as LEAF Node. In simple words, a leaf is a node with no child.

In a tree data structure, the leaf nodes are also called as External Nodes. External node is also a node with no child. In a tree, leaf node is also called as 'Terminal' node. Ex:

(K,L,F,G,M,I,J)

7. Degree In a tree data structure, the total number of children of a node (or) number of subtrees of a node is called as DEGREE of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as 'Degree of Tree'

8. Level

In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple

words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step). Some authors start root level with 1.

9. Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest

path is called as HEIGHT of that Node. In a tree, height of the root node is said to be height of the

tree. In a tree, height of all leaf nodes is '0'.

10. Depth

In a tree data structure, the total number of edges from root node to a particular node is called

as DEPTH of that Node. In a tree, the total number of edges from root node to a leaf node in the

longest path is said to be Depth of the tree. In simple words, the highest depth of any leaf node in a

tree is said to be depth of that tree. In a tree, depth of the root node is '0'.

11. Path In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as PATH between that two Nodes. Length of a Path is total number of nodes in that path. In below example the path A - B - E - J has length 4.

12. Sub Tree In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.

13. ancestor: An ancestor of a node in a binary tree is any node that lies on the path from the root node to that particular node. Ancestor nodes are higher in the tree hierarchy and can be found by tracing the path from the root to the target node.

14. Binary search tree inserting.

The very first insertion creates the tree. Afterwards, whenever an element is to be inserted. First locate its proper location. Start search from root node then if data is less than key value, search empty location in left sub tree and insert the data. Otherwise search empty location in right sub tree and insert the data.

In a binary search tree, the insertion operation is performed with $O(\log n)$ time complexity. In binary search tree, new node is always inserted as a leaf node. The insertion operation is performed as follows...

Step 1: Create a newNode with given value and set its left and right to NULL.

Step 2: Check whether tree is Empty.

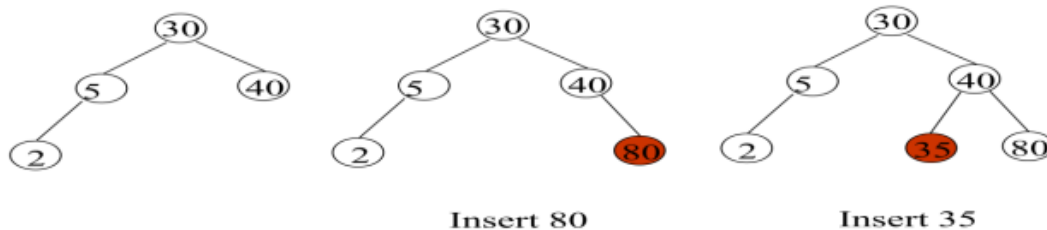
Step 3: If the tree is Empty, then set set root to newNode.

Step 4: If the tree is Not Empty, then check whether value of newNode is smaller or larger than the node (here it is root node).

Step 5: If newNode is smaller than or equal to the node, then move to its left child. If newNode is larger than the node, then move to its right child.

Step 6: Repeat the above step until we reach a node (e.i., reach to NULL) where search terminates.

Step 7: After reaching a last node, then insert the newNode as left child if newNode is smaller or equal to that node else insert it as right child.



15. Complete Binary Tree

In a binary tree, every node can have a maximum of two children. But in strictly binary tree, every node should have exactly two children or none and in complete binary tree all the nodes must have exactly two children and at every level of complete binary tree there must be 2 level number of nodes. For example at level 2 there must be $2^2 = 4$ nodes and at level 3 there must be $2^3 = 8$ nodes.

16. link representation of a binary tree in memory?

Binary trees in linked representation are stored in the memory as linked lists. These lists have nodes that aren't stored at adjacent or neighboring memory locations and are linked to each other through the parent-child relationship associated with trees.

In this representation, each node has three different parts –

- pointer that points towards the right node,
- pointer that points towards the left node,
- data element.

This is the more common representation. All binary trees consist of a root pointer that points in the direction of the root node. When you see a root node pointing towards null or 0, you should know that you are dealing with an empty binary tree. The right and left pointers store the address of the right and left children of the tree.

17. Binary search tree .

A Binary Search Tree (BST) is a binary tree data structure in which each node has at most two children, referred to as the left child and the right child.

18.how can binary search tree overcome the disadvantage of array and linked list

Binary Search Trees (BSTs) offer efficient search ($O(\log n)$) and dynamic sizing like linked lists, overcoming array limitations. Unlike arrays, they allow ordered storage. However, BSTs can suffer from worst-case performance and higher memory overhead due to pointers. Self-balancing BSTs like AVL and Red-Black trees mitigate worst-case scenarios, maintaining efficient operations. Overall, BSTs provide a balance between search efficiency, dynamic sizing, and ordered storage, making them advantageous in many scenarios.

19.what is an extended tree

An "extended tree" is not a standard term in computer science or data structures. It doesn't have a widely recognized or defined meaning in the context of trees or tree data structures.

20.illustrate similar and copies with example

****Similarity****: In the world of music, two songs with similar melodies or rhythms, like "Let It Be" by The Beatles and "No Woman, No Cry" by Bob Marley, share a musical similarity due to their soothing, ballad-like qualities.

****Copies****: In literature, a copy of a book might be an identical reprint of the original, preserving the same text and content as the first edition. However, a revised edition of a book is a modified copy that includes updates, corrections, or additional material, like an updated version of a classic novel with an author's commentary.

21.Preorder Traversal,Postorder Traversal,inorder traversal.

Preorder Traversal

1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left->subtree)
3. Traverse the right subtree, i.e., call Preorder(right->subtree)

Postorder Traversal

1. Traverse the left subtree, i.e., call Postorder(left->subtree)
2. Traverse the right subtree, i.e., call Postorder(right->subtree)
3. Visit the root

inorder traversal

1. Traverse the left subtree, i.e., call Inorder(left->subtree)
2. Visit the root.
3. Traverse the right subtree, i.e., call Inorder(right->subtree)

21. advantage of binary search tree.

Advantages of Binary Search Tree:

- BST is fast in insertion and deletion when balanced. ...
- BST is also for fast searching, with a time complexity of $O(\log n)$ for most operations.
- BST is efficient. ...
- We can also do range queries – find keys between N and M ($N \leq M$).
- BST code is simple as compared to other data structures.

